

KaamYaar — Technical Requirements Document (TRD)

Version: 1.0

Status: Pre-Build

1. Purpose

Define the technical implementation requirements for KaamYaar.

This document converts product requirements into engineering decisions.

Primary goals:

- Scalability
 - Reliability
 - Security
 - Maintainability
 - Fast iteration
-

2. System Overview

KaamYaar consists of:

1. Customer Mobile App
2. Worker Mobile App
3. Admin Dashboard
4. Backend Services
5. Notification System
6. Location System

Architecture Style:

Clean Architecture

Mobile-first

Realtime-enabled

3. Technical Stack

Frontend

Flutter

Responsibilities:

- UI
 - State
 - Navigation
 - Device APIs
-

Backend

Supabase

Responsibilities:

- Auth
 - Database
 - Storage
 - Realtime
 - Policies
-

Database

PostgreSQL

Notifications

Firebase Cloud Messaging

Maps

Google Maps

State Management

Riverpod

Navigation

GoRouter

4. Environment Architecture

Environments:

Development

Staging

Production

Rules:

No production secrets locally.

Separate backend per environment.

Example:

kaamyaar-dev

kaamyaar-stage

kaamyaar-prod

5. Repository Structure

Root:

kaamyaar/

Folders:

docs/

mobile/

backend/

scripts/

Flutter:

lib/

core/

features/

auth/

services/

booking/

tracking/

worker/

profile/

Each feature contains:

data/

domain/

presentation/

6. Mobile Architecture

Pattern:

Presentation

↓

Use Case



Repository



Backend

Rules:

UI never talks directly to database.

Business rules stay in domain layer.

Repositories own data access.

7. Authentication

Method:

Phone OTP

Flow:

Phone



OTP



Verify



Create Session



Role Resolution

↓

Route

Requirements:

OTP expiration

Session persistence

Secure token storage

Refresh recovery

8. Authorization

Roles:

customer

worker

admin

Access:

Customer:

Own bookings only

Worker:

Assigned bookings only

Admin:

Full access

Implementation:

Backend policy enforcement

9. Database Requirements

Database:

PostgreSQL

Primary Tables:

users

services

workers

addresses

bookings

booking_photos

reviews

payments

worker_locations

notifications

Requirements:

UUID keys

Soft deletes

Created timestamps

Indexes

10. Realtime Architecture

Events:

booking_created

booking_assigned

booking_accepted

location_updated

booking_completed

Mechanism:

Database Event

↓

Realtime

↓

Client Listener

↓

UI Update

Rules:

Reconnect automatically.

Prevent duplicate processing.

11. Worker Assignment Engine

Input:

Service

Location

Availability

Verification

Process:

Filter:

approved

↓

online

↓

same category

↓

distance

↓

assign

Fallback:

Expand search radius.

Requirements:

Assignment <10 seconds

12. Location System

Sources:

GPS

Rules:

Track only active jobs.

Update:

5 seconds

Store:

latitude

longitude

timestamp

Recovery:

Retry failed updates.

13. Media Storage

Provider:

Supabase Storage

Buckets:

profile-images

worker-documents

booking-photos

Rules:

Image compression

Size limits

Validation

14. Notification Architecture

Triggers:

booking_created

worker_assigned

worker_arrived

payment_completed

Flow:

Event



Notification Queue



Push



Device

Requirements:

Retry delivery.

Track failures.

15. API Requirements

Pattern:

Repository



Service



Backend

Requirements:

Timeouts

Retry

Versioning

Validation

Idempotency

16. Error Handling

Categories:

Network

Auth

Booking

Location

Storage

Every request returns:

Success

Failure

Loading

Rules:

No raw exceptions.

17. Offline Behavior

Requirements:

Cache sessions

Queue actions

Recover network

Sync automatically

Offline allowed:

View history

Profile

Offline blocked:

Create booking

Payment

18. Security Requirements

Controls:

JWT

RLS

Input Validation

Rate Limiting

Secret Management

Rules:

No secret keys inside app.

Encrypt sensitive data.

19. Performance Targets

Cold Start:

<3 sec

Screen Transition:

<300 ms

Booking Creation:

<2 sec

Assignment:

<10 sec

Crash Rate:

<1%

20. Monitoring

Track:

Errors

Crashes

Latency

Requests

Realtime health

Alerts:

Authentication failures

Assignment failures

21. Testing Strategy

Unit Tests

Widget Tests

Integration Tests

Manual QA

Coverage Goal:

70%+

Critical:

Booking

Auth

Tracking

22. Release Strategy

Flow:

Dev

↓

Internal

↓

Beta

↓

Production

Rollback:

Mandatory

23. Deployment Rules

No direct production deploy.

Require:

Testing

Review

Migration approval

Monitoring

24. Production Readiness Checklist

Authentication

Database

Policies

Realtime

Notifications

Tracking

Analytics

Monitoring

Backup

Disaster Recovery

Final Engineering Rule

If code violates:

Reliability

Security

Maintainability

Reject implementation.